8 Verifiche di uguaglianza

Vediamo ora due algoritmi *Monte Carlo* molto efficienti basati su semplici proprietà algebriche. **Entrambi gli algoritmi utilizzano una funzione casuale M che verifica l'uguaglianza tra due elementi x e y appartenenti a un universo U grande, a partire dalle loro immagini $M(x)$ ed $M(y)$, o *fingerprint*, appartenenti a un universo V molto più piccolo**.

Controllo della moltiplicazione fra matrici

L'algoritmo per moltiplicare due matrici $n \times n$ $\mathbf{A}$ e $\mathbf{B}$ richiede $O(n^3)$ moltiplicazioni. Se $\mathbf{C} = \mathbf{AB}$, infatti, servono n moltiplicazioni per calcolare ognuno degli $n \times n$ elementi

$$C_{ij} = \sum_{k=1}^{n} A_{ik} B_{kj}$$

Nel caso di matrici 2×2, tuttavia, si può ottenere lo stesso risultato con 7 moltiplicazioni ponendo

$$\begin{aligned} I &= (a_{11} + a_{22})(b_{11} + b_{22}) & II &= (a_{21} + a_{22})\, b_{11} \\ III &= a_{11}(b_{12} - b_{22}) & IV &= a_{22}(-b_{11} + b_{21}) \\ V &= (a_{11} + a_{12})\, b_{22} & VI &= (-a_{11} + a_{21})(b_{11} + b_{12}) \\ VII &= (a_{12} - a_{22})(b_{21} + b_{22}) \end{aligned}$$

troviamo infatti che

$$\begin{aligned} I + IV - V + VII &= a_{11}b_{11} + a_{12}b_{21} = c_{11} & III + V &= a_{11}b_{12} + a_{12}b_{22} = c_{12} \\ II + IV &= a_{21}b_{11} + a_{22}b_{21} = c_{21} & I + III - II + VI &= a_{21}b_{12} + a_{22}b_{22} = c_{22} \end{aligned}$$

Questa riscrittura è alla base di un algoritmo ricorsivo che richiede $O(n^{\log_2 7}) = O(n^{2.807})$ moltiplicazioni [9]. Negli anni l'esponente è stato ripetutamente abbassato. Al momento l'algoritmo ottimale richiede $O(n^{2.372})$ moltiplicazioni, ma la verifica di una sua corretta implementazione è estremamente complicata. Per n grande, è conveniente utilizzare un algoritmo *Monte Carlo* capace di verificare l'uguaglianza **matriciale**

$$\mathbf{AB} = \mathbf{C} \tag{11}$$

con $O(n^2)$ moltiplicazioni necessarie a verificare

$$\mathbf{ABr} = \mathbf{Cr}$$

dove $\mathbf{r}$ è un **vettore** n-dimensionale casuale. Vediamo prima l'algoritmo e poi discutiamo la sua notevole efficienza.

Algoritmo 8.1. *MCMatrixMultiplicationVerifier*$(\mathbf{A}, \mathbf{B}, \mathbf{C})$
Input: $\mathbf{A}$, $\mathbf{B}$ e $\mathbf{C}$ matrici $n \times n$
Output: $\mathbf{AB} \neq \mathbf{C}$ o `probabilmente` $\mathbf{AB} = \mathbf{C}$

---

1. campiona $\mathbf{r}$ uniformemente a caso in $\{0,1\}^n$
2. $\mathbf{s} \leftarrow \mathbf{Br}$ # $\mathbb{P}(r_j = 0) = \mathbb{P}(r_j = 1) = 1/2$ per $j = 1, \ldots, n$
3. $\mathbf{t} \leftarrow \mathbf{As}$ # $\mathbf{t} = \mathbf{A}(\mathbf{Br})$
4. $\mathbf{u} \leftarrow \mathbf{Cr}$
5. `if` $\mathbf{t} = \mathbf{u}$
   `return probabilmente` $\mathbf{AB} = \mathbf{C}$
   `else`
   `return` "$\mathbf{AB} \neq \mathbf{C}$"

Osservazione 8.1. *Da 3 a 2*
I tre vettori n-dimensionali $\mathbf{s}$, $\mathbf{t}$ e $\mathbf{u}$ sono calcolati come prodotti di una matrice $n \times n$ con un vettore n-dimensionale e richiedono quindi solo $O(n^2)$ moltiplicazioni ciascuno.

Osservazione 8.2. *Una questione di associatività*
MCMatrixMultiplicationVerifier$(\mathbf{A}, \mathbf{B}, \mathbf{C})$ si fonda sul fatto che, sfruttando la proprietà associativa del prodotto, l'azione della matrice $\mathbf{AB}$ su un vettore $\mathbf{r}$ può essere calcolata a partire dalle matrici $\mathbf{A}$ e $\mathbf{B}$ in cascata. Infatti, abbiamo che

$$(\mathbf{AB})\mathbf{r} = \mathbf{A}(\mathbf{Br})$$

□

Con che probabilità *MCMatrixMultiplicationVerifier*$(\mathbf{A}, \mathbf{B}, \mathbf{C})$ restituisce `probabilmente` $\mathbf{AB} = \mathbf{C}$, se $\mathbf{AB} \neq \mathbf{C}$?

Proposizione 8.1. *MCMatrixMultiplicationVerifier*$(\mathbf{A}, \mathbf{B}, \mathbf{C})$ *è un algoritmo Monte Carlo*
Se $\mathbf{AB} \neq \mathbf{C}$, allora

$$\mathbb{P}(\mathbf{A}(\mathbf{Br}) = \mathbf{Cr}) \leq \frac{1}{2}$$

Dimostrazione: se $\mathbf{AB} \neq \mathbf{C}$ almeno un elemento della matrice $\mathbf{D} = \mathbf{AB} - \mathbf{C}$ deve essere non nullo. Supponiamo che sia l'elemento d_{ij} di riga i e colonna j. In questo caso $\mathbf{A}(\mathbf{Br}) = \mathbf{Cr}$ può verificarsi solo se la moltiplicazione dell'i-esima riga di $\mathbf{D}$ con il vettore colonna casuale $\mathbf{r}$ restituisce 0, ovvero se $\sum_k d_{ik} r_k = 0$. Isolando il termine che contiene d_{ij} e ponendo $y = \sum_{k \neq j} d_{ik} r_k$ otteniamo

$$\sum_k d_{ik} r_k = d_{ij} r_j + \sum_{k \neq j} d_{ik} r_k = d_{ij} r_j + y$$

Possiamo quindi riscrivere la probabilità di errore $\mathbb{P}(\mathbf{A}(\mathbf{Br}) = \mathbf{Cr})$ con $\mathbb{P}(d_{ij} r_j + y = 0)$. Poiché $\mathbb{P}(y = 0 \cup y \neq 0) = 1$ sappiamo che $\mathbb{P}(d_{ij} r_j + y = 0)$ può essere espressa come probabilità totale, ovvero

$$\mathbb{P}(d_{ij} r_j + y = 0) = \mathbb{P}(d_{ij} r_j + y = 0 | y = 0)\mathbb{P}(y = 0) + \mathbb{P}(d_{ij} r_j + y = 0 | y \neq 0)\mathbb{P}(y \neq 0)$$

Ma

$$\mathbb{P}(d_{ij} r_j + y = 0 | y = 0) = \mathbb{P}(d_{ij} r_j = 0) = \mathbb{P}(r_j = 0) = \frac{1}{2}$$

e

$$\mathbb{P}(d_{ij} r_j + y = 0 | y \neq 0) = \mathbb{P}(r_j = 1 \cap d_{ij} = -y) \leq \mathbb{P}(r_j = 1) = \frac{1}{2}$$

da cui otteniamo

$$\mathbb{P}r(\mathbf{A}(\mathbf{Br}) = \mathbf{Cr}) = \mathbb{P}(d_{ij} r_j + y = 0) \leq \frac{1}{2}\mathbb{P}(y = 0) + \frac{1}{2}\mathbb{P}(y \neq 0) = \frac{1}{2}$$

■

Forti di questo risultato, eseguendo *MCMatrixMultiplicationVerifier*$(\mathbf{A}, \mathbf{B}, \mathbf{C})$ k volte, possiamo quindi verificare la validità dell'uguaglianza (11) con probabilità

$$\mathbb{P}(k) = 1 - \frac{1}{2^k}$$

Compito 8.1. *Implementazione di MCMatrixMultiplicationVerifier*$(\mathbf{A}, \mathbf{B}, \mathbf{C})$
Come prima cosa, sincerati di sapere rispondere alle domande che trovi in fondo alle note relative a questo capitolo. Costruisci $\mathbf{A}$ e $\mathbf{B}$ matrici 100×100 campionando uniformemente dall'insieme $\{-2, -1, 0, 1, 2\}$. Calcola $\mathbf{C} = \mathbf{AB}$ e aggiungi e sottrai un'unità a due elementi di una stessa riga (C_{22} e C_{28} ad esempio). Implementa *MCMatrixMultiplicationVerifier*$(\mathbf{A}, \mathbf{B}, \mathbf{C})$ e lancialo 100 volte. Consegna un elaborato in cui descrivi a parole **(a)** il problema che hai affrontato, **(b)** il funzionamento di *MCMatrixMultiplicationVerifier*$(\mathbf{A}, \mathbf{B}, \mathbf{C})$ e **(c)** i risultati che hai ottenuto per $k = 2, 5$ e 10.

Confronto di file o stringhe binarie

Supponiamo che Bob voglia verificare che il file B in suo possesso sia uguale al file A di Alice, entrambi di ℓ bit. Nel caso di file grandi, la verifica può essere dispendiosa soprattutto per il tempo necessario per la trasmissione del file A a Bob (problema della sincronizzazione di copie di uno stesso file). Vediamo come un algoritmo *Monte Carlo* consenta di verificare l'uguaglianza di A e B trasmettendo solo $2 \ln \ell$ bit.

Ci serve un risultato fondamentale della teoria dei numeri che enunciamo senza dimostrazione.

Teorema 8.1. *Numerosità asintotica dei numeri primi*
Indichiamo con $\pi(x)$ il numero di primi minori o uguali a $x \in \mathbb{R}^+$ e con $\ln x$ il logaritmo naturale di x. Si ha che

$$\lim_{x \to \infty} \frac{\pi(x)}{x/\ln x} = 1$$

□

La tabella di seguito mostra alcuni valori e chiarisce come la convergenza a 1 del rapporto al crescere di x non equivale alla convergenza del numeratore al denominatore

x	$\pi(x)$	$\pi(x) - x/\ln x$	$\pi(x)/(x/\ln x)$
10	4	-0.3	0.921
10^3	168	23	1.161
10^6	78,498	6116	1.084
10^{12}	37,607,912,018	1,416,705,193	1.039
10^{23}	1,925,320,391,606,818,000,000	37,083,513,766,592,670,000	1.020

Abbiamo bisogno, infine, anche di un risultato molto semplice.

Proposizione 8.2. *Maggiorazione del numero di fattori primi*
I fattori primi di un qualunque numero $n < 2^\ell$, per ogni $\ell \in \mathbb{N}$, sono al più ℓ.
Dimostrazione: tutti i numeri primi sono maggiori o uguali a 2. Se n avesse più di ℓ fattori primi, avremmo che $n > 2^\ell$. ■

Definizione 8.1. *Impronta digitale*
Sia p un numero primo compreso tra 2 e ℓ^2 ed $n \in \mathbb{N}$ con $n < 2^\ell$. L'*impronta digitale* $f(n)$ di n è definita come

$$n \pmod p$$

□

L'idea di *MCStringEqualityVerifier*(a, b) è che Alice legge il file A come il numero naturale a. Poiché il file A è formato da al più ℓ bit, abbiamo che $a \leq 2^\ell$. Poiché $p < \ell^2$, invece, $f(n)$ richiede solo $2 \ln \ell$ bit. Alice, pertanto, calcola $f(a)$ e spedisce a Bob $2 \ln \ell$ bit invece che ℓ bit. Bob, per parte sua, legge il file B come il numero naturale b e confronta $f(a)$ con $f(b)$.

Algoritmo 8.2. *MCStringEqualityVerifier*(a, b)
Input: a e b stringhe binarie di lunghezza ℓ
Output: `input diversi` o `input probabilmente uguali`

---

1. campiona p, primo, uniformemente a caso in $\{2, \ldots, \ell^2\}$
2. $f(a) \leftarrow a \pmod p$
3. $f(b) \leftarrow b \pmod p$
4. `if` $|f(a) - f(b)| \equiv 0 \pmod p$
   `return input probabilmente uguali`
   `else`
   `return input diversi`

Il campionamento del numero primo p potrebbe essere basato o su una lista pre-calcolata di numeri primi o sulla ripetizione del *MCMillerRabinTest* per determinare la primalità di un numero p campionato uniformemente in $\{2, \ldots, \ell^2\}$.

Valutiamo ora la probabilità d'errore per *MCStringEqualityVerifier*(a, b).

Proposizione 8.3. *MCStringEqualityVerifier*(a, b) *è un algoritmo Monte Carlo molto efficiente*
Se $\mathcal{P}(\ell)$ è la probabilità che l'algoritmo **8.2** restituisca `input probabilmente uguali` quando $a \neq b$, abbiamo

$$\mathcal{P}(\ell) \approx \frac{2 \ln \ell}{\ell}$$

Dimostrazione: Se $a \neq b$ allora $1 \leq |a - b| \leq 2^\ell$. Per la proposizione **8.2**, allora, $|a - b|$ ha al più ℓ fattori primi e, poiché dal teorema **8.1** sappiamo che i numeri primi tra 1 e ℓ^2 sono dell'ordine di $\ell^2 / 2 \ln \ell$, per la probabilità di errore abbiamo

$$\mathcal{P}(\ell) \approx \frac{\ell}{\ell^2 / 2 \ln \ell} = \frac{2 \ln \ell}{\ell}$$

■

Osservazione 8.3. *Niente male per un Monte Carlo*
La probabilità di errore di *MCStringEqualityVerifier*(a, b) nel caso di ℓ grandi è decisamente piccola. Se dovessimo confrontare file dell'ordine di Gbit, ovvero $\ell = 2^{30}$ bit, otterremmo

$$\mathcal{P}\left(2^{30}\right) \approx \frac{2 \ln 2^{30}}{2^{30}} = \frac{60 \ln 2}{2^{30}} < \frac{2^6}{2^{30}} = 2^{-24} < 10^{-7}$$

Compito 8.2. *File piccoli ma non troppo piccoli*
Come prima cosa, sincerati di sapere rispondere alle domande che trovi in fondo alle note relative a questo capitolo. Considera un file a di 30 bit e un numero primo p a caso tra 2 e 900. Determina tutti i possibili file b per i quali l'output di *MCStringEqualityVerifier*(a, b), nell'ipotesi in cui il numero primo campionato sia sempre p, è `input probabilmente uguali`. Consegna un elaborato in cui descrivi a parole **(a)** il problema che hai affrontato, **(b)** il funzionamento di *MCStringEqualityVerifier*(a, b) e **(c)** i risultati che hai ottenuto. Calcola infine la probabilità con la quale il numero primo da te utilizzato sia lo stesso usato in un'altra relazione.